# Docker e Containers: Deploy e Hospedagem

Conteinerização, construção, execução, deploy e modelos de hospedagem




15 / 10 / 2026

# Docker, deploy e hospedagem: três papéis diferentes

### DOCKER

Empacota e executa aplicações em containers.

### DEPLOY

Disponibiliza uma versão em um ambiente.

### HOSPEDAGEM

Mantém a aplicação acessível e operante.

Uma imagem local não se torna pública sozinha: é preciso publicar, executar e manter infraestrutura acessível.

# “Funciona na minha máquina.”

O container reduz divergências — não elimina o mundo ao redor.

```
$ node server.js
Error: ABI incompatível
Node 18 ≠ módulo compilado para Node 20
```



VERSÕES • DEPENDÊNCIAS • VARIÁVEIS

KERNEL • ARQUITETURA • PERMISSÕES

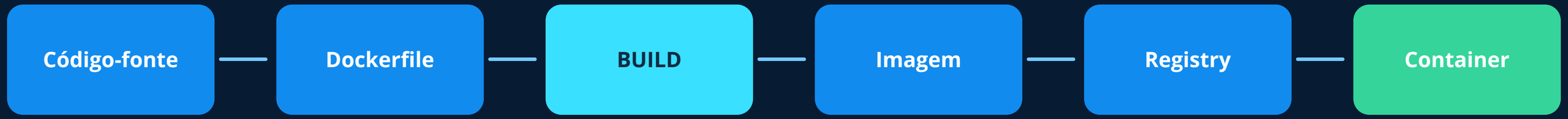
# Containers × máquinas virtuais




VM: isolamento forte, SO convidado, mais recursos
Container: kernel compartilhado, isolamento de processo, inicialização rápida

# Do código ao container em execução

CLI e Engine coordenam construção, distribuição e execução.




```
Docker CLI  →  comandos do usuário

Docker Engine  →  build + execução
```

# Comandos fundamentais

```
$ docker pull nginx:alpine
$ docker run -d --name web -p 8080:80 nginx:alpine
$ docker ps
$ docker logs web
$ docker stop web
$ docker rm web
```

**8080** → **80**

porta do host — porta do container

Remover o container apaga a instância. A imagem continua disponível até ser removida separadamente.

# Dockerfile: a receita da imagem

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
ENV NODE_ENV=production
EXPOSE 3000
CMD ["node", "server.js"]
```

RUN
executa durante o build

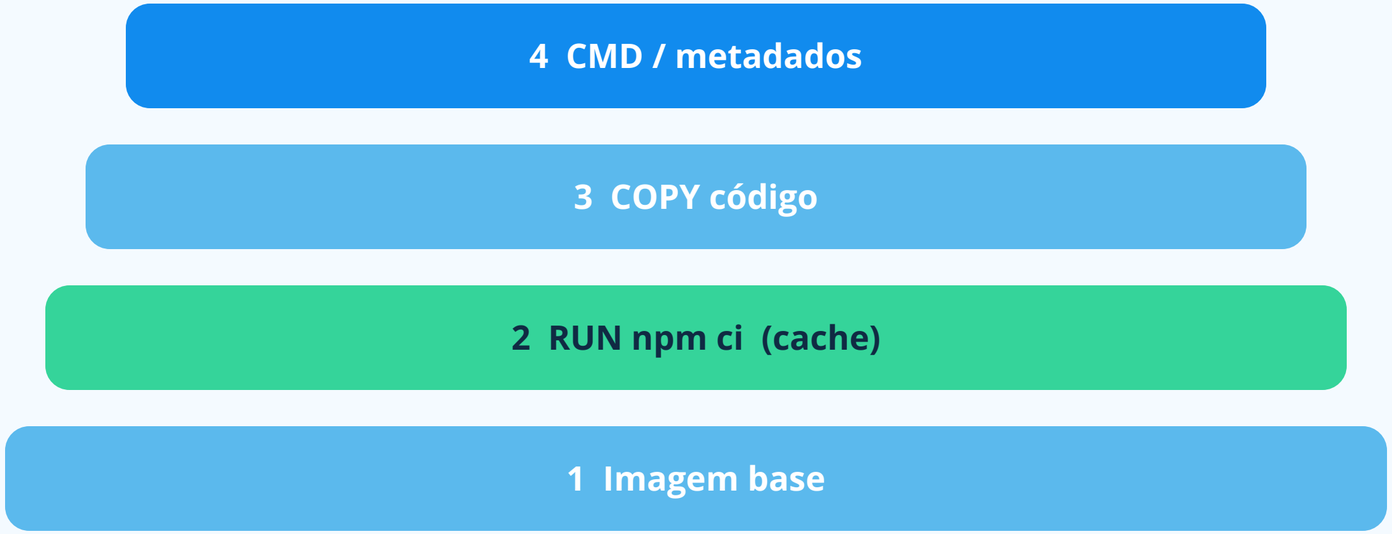
CMD
comando padrão na execução

EXPOSE
documenta a porta esperada

# Build eficiente: contexto, cache e tags

```
docker build -t minha-api:1.0 .
```

```
.dockerignore
.git
node_modules
.env
```




O ponto final define o contexto. Envie apenas o necessário; organize instruções estáveis primeiro para reaproveitar camadas em cache.

# Para sair do demo e chegar à produção

**REDE**
Portas mínimas e exposição segura

**AMBIENTE**
Configuração dinâmica sem rebuild

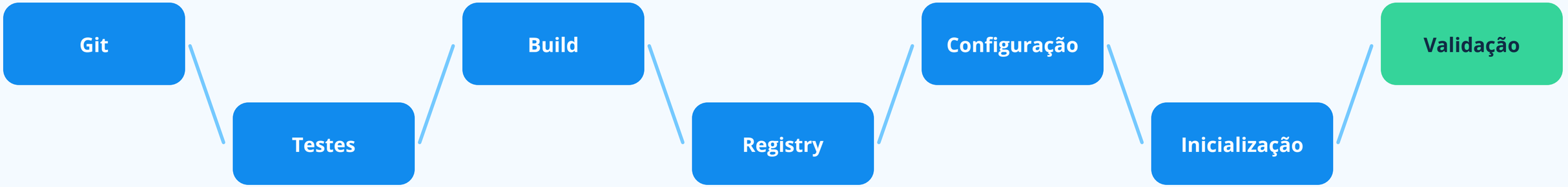
**VOLUMES**
Persistência além do container

**COMPOSE**
Múltiplos serviços como um sistema

```
services:
  api:
    build: .
    ports: ["8080:3000"]
    environment:
      - NODE_ENV=production
    volumes:
      - dados-app:/data
```

```
docker compose up -d --build
```

# Deploy é um processo — não um botão




**Deploy concluído ≠ aplicação saudável. Logs, métricas e health checks fecham o ciclo.**

# Escolha o modelo pela carga de trabalho

### ESTÁTICO

HTML, CSS e JS
Entrega por CDN
Sem backend persistente

### SERVERLESS

Função sob demanda
Curta duração
Escala por invocação

### PERSISTENTE

API ou worker
Processo contínuo
Rede e disco configuráveis

Estado, armazenamento, rede e escala mudam de acordo com o modelo.

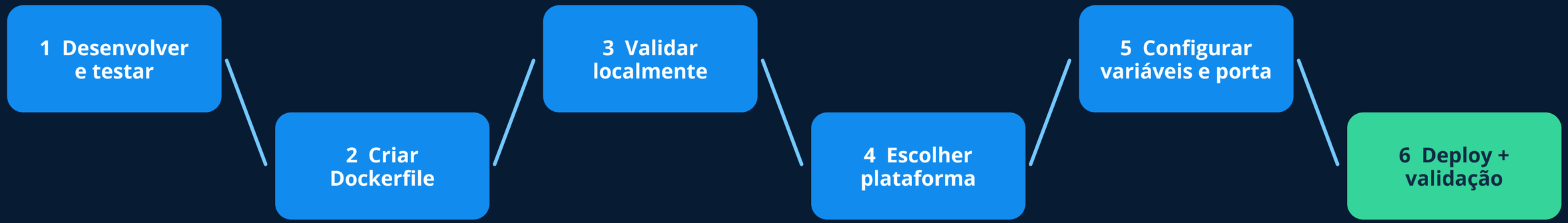
# Plataformas: o modelo predominante importa

| | | |
|---|---|---|
| **VERCEL** | Web + funções | OCI como função |
| **NETLIFY** | Estático + serverless | Fluxo orientado a sites |
| **HEROKU** | Aplicação gerenciada | Container Registry |
| **RAILWAY** | Serviço persistente | Dockerfile ou imagem |
| **GITHUB PAGES** | Hospedagem estática | Sem backend containerizado |

# Caso: publicar uma API Node.js

Requisitos

HTTP • variáveis • logs • banco futuro




```
GET /health  →  200 OK
```

# Produção confiável exige disciplina contínua

**BASES**
confiáveis e atualizadas

**PRIVILÉGIOS**
mínimos

**SEGURA**
**EFICIENTE**
**OPERÁVEL**

**SEGREDOS**
fora do Dockerfile

**OBSERVAR**
logs, métricas, health

**RECUPERAR**
backup e rollback

# Reproduzir. Publicar. Operar.

Docker padroniza ambientes com imagens e containers. Deploy e hospedagem completam o ciclo — e a plataforma certa nasce dos requisitos técnicos, não da moda.

**"O objetivo profissional não é apenas fazer a aplicação funcionar no computador ou publicá-la na internet, mas construir um processo de entrega reproduzível, seguro, verificável e sustentável."**

Documentação oficial: Docker • Vercel • Railway • provedores de nuvem

## Obrigado pela atenção!